ТЕХНИЧЕСКИЙ ОТЧЕТ И АРХИТЕКТУРНАЯ СПЕЦИФИКАЦИЯ

МУЛЬТИАГЕНТНАЯ SAAS-ПЛАТФОРМА АВТОНОМНОГО КОНТЕНТ-МАРКЕТИНГА «SOCIALIX»

---

1. ОБЗОР СИСТЕМЫ И ИЕРАРХИЧЕСКАЯ АРХИТЕКТУРА (HUB & SPOKE)

1.1 Назначение и миссия платформы

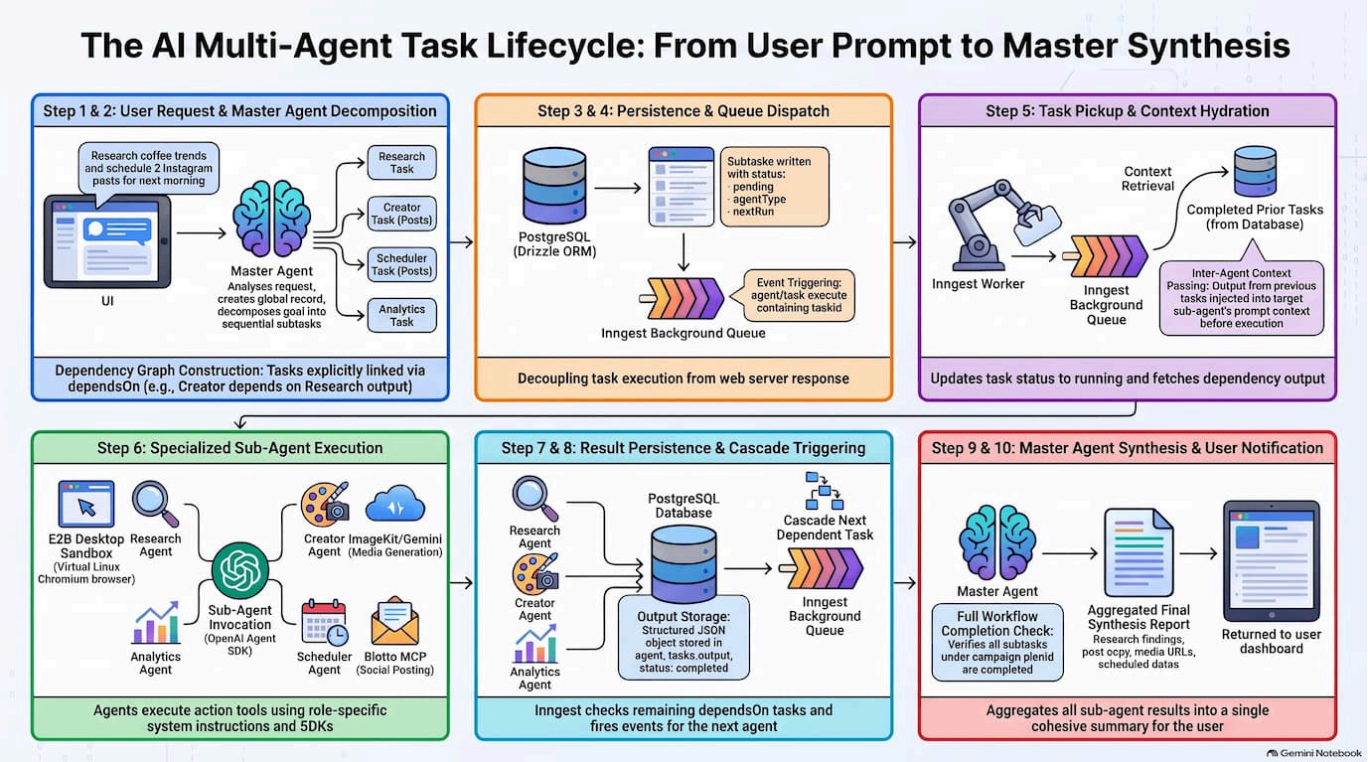
Socialix представляет собой полнофункциональную SaaS-платформу на базе искусственного интеллекта, предназначенную для сквозной автоматизации контент-маркетинга в социальных сетях (Instagram, TikTok, YouTube, X, LinkedIn). Платформа оркестрирует виртуальную команду специализированных ИИ-агентов под управлением центрального управляющего агента (**Master Agent**).

Вместо ручного поиска трендов, написания текстов, создания и обработки визуалов и ручной выгрузки постов, пользователь задает одну верхнеуровневую команду на естественном языке (например: *«Исследовать тренды и запланировать 2 поста в Instagram на запуск новой кофейни на следующую неделю на 8:00»*). Мультиагентная система автоматически декомпозирует запрос, проводит веб-исследования в виртуальном Linux-браузере, генерирует графические и текстовые материалы, формирует контент-календарь, публикует посты и анализирует показатели вовлеченности.

1.2 Архитектурный паттерн Hub & Spoke и ролевая модель

Система построена на базе иерархического паттерна **Hub & Spoke («Менеджер — Сотрудник»)**. Прямая хаотичная коммуникация между агентами «каждый с каждым» отсутствует; вместо этого вся координация и передача данных осуществляются либо через Master Agent, либо асинхронно через persistent-слой базы данных PostgreSQL.

Графическая схема архитектуры Socialix Platform



Текстовое описание схемы архитектуры:

1. **User Request (1):** Пользователь отправляет текстовую команду через веб-интерфейс (Next.js & React / FastJS Template).
2. **Master Agent (2):** Создает глобальный план (`agent_plans`) в PostgreSQL через Drizzle ORM и разбивает его на подзадачи (`agent_tasks`).
3. **Assign Task (3):** Задачи распределяются в фоновую очередь Inngest (`queueScheduledAgentTask` / `executeAgentTask`).
4. **Research Agent (4):** Подключается к изолированной песочнице **E2B Desktop Sandbox** (Linux Chromium VM) для поиска трендов и парсинга страниц.
5. **Creator Agent (5):** Использует Gemini AI и ImageKit API для написания постов и AI-обработки графики (удаление фона, кадрирование).
6. **Scheduler Agent (6):** Отправляет готовые публикации через **Blotato API / MCP Tools** в Instagram, TikTok, YouTube, X.
7. **Analytics Agent (7):** Запрашивает метрики вовлеченности через Blotato Analytics MCP и передает их обратно Master Agent'y.

Роли и зоны ответственности в мультиагентной команде:

1. **Master Agent (Главный менеджер / Оркестратор):**
   - **Назначение:** Руководитель виртуального офиса.
   - **Функции:** Принимает верхнеуровневый запрос пользователя, создает стратегический план в таблице `agent_plans` , разбивает единую цель на цепочку подзадач в `agent_tasks` с учетом графа зависимостей (`dependsOn`), отслеживает статусы их выполнения и агрегирует выходы (`output`) всех субагентов в итоговый отчет.
2. **Research Agent (Маркетолог-исследователь):**

- **Назначение:** Исследование ниши, трендов и конкурентов.
- **Функции:** Формирует поисковые запросы, парсит веб-страницы, задействует изолированную облачную Linux-песочницу **E2B Desktop Sandbox** с активным виртуальным браузером Chromium для безопасного обхода защит и парсинга динамического контента.

3. **Creator Agent (Создатель контента и визуальный дизайнер):**
   - **Назначение:** Копирайтинг и AI-генерация/обработка медиафайлов.
   - **Функции:** Пишет цепляющие заголовки (Hook), основной текст поста (Body), призывы к действию (CTA) и массив хэштегов. Генерирует изображения через Gemini/ImageKit API и выполняет программатические AI-трансформации (удаление фона, смарт-кроппинг, изменение размеров, наложение текста).
4. **Scheduler Agent (Планировщик и контент-менеджер):**
   - **Назначение:** Управление контент-календарем и выгрузка публикаций.
   - **Функции:** Сопоставляет время выгрузки с часовыми поясами целевой аудитории, форматирует материалы под спецификации платформ и вызывает инструменты **Blotato (Blotto) API / MCP** для мгновенной или отложенной публикации.
5. **Analytics Agent (Аналитик данных):**
   - **Назначение:** Оценка эффективности и сбор метрик.
   - **Функции:** Запрашивает данные по просмотрам, лайкам, комментариям и вовлеченности за 7 или 30 дней через Blotato MCP Tools. Формирует выводы и передает их Master Agent для корректировки будущих планов.

1.3 Ключевой технологический стек

- **Frontend & Framework:** Next.js (App Router), React, Tailwind CSS, Shadcn UI (базовый каркас на FastJS).
- **AI Orchestration & SDK:** OpenAI Agent SDK, Zod (схемы валидации), Claude AI (расширение Claude Code для VS Code с маршрутизацией простых задач на Haiku, сложных — на Opus 3.5).
- **Фоновая оркестрация и очереди:** Inngest (cron каждые 15 минут, фоновые воркеры, контроль параллелизма).
- **База данных и ORM:** PostgreSQL (инфраструктура InsForge / Supabase / Neon) + Drizzle ORM.
- **Изолированные среды выполнения (Execution Sandboxes):**
  - **E2B Desktop Sandbox:** Облачные виртуальные машины Linux с графическим окружением для веб-парсинга.
  - **ImageKit API:** AI-генерация, удаление фона, наложение текста и оптимизация изображений.
  - **Blotato (Blotto) API & MCP Server:** Единый шлюз для публикации и сбора аналитики в Instagram, TikTok, YouTube, X, LinkedIn.
- **CI/CD и AI Code Review:** CodeRabbit (автоматический анализ пулл-реквестов, статический сканер безопасности).

---

2. ПОЛНАЯ СХЕМА БАЗЫ ДАННЫХ (DRIZZLE ORM)

Управление состоянием кампаний, задач, аудита и межагентных коммуникаций реализовано в PostgreSQL через Drizzle ORM (`db/schema.ts`).

2.1 Полный исходный код схемы: `db/schema.ts`

```
import {
  pgTable,
  text,
  timestamp,
  integer,
  jsonb,
  uuid
} from 'drizzle-orm/pg-core';

// 1. Профили пользователей
export const users = pgTable('users', {
  id: uuid('id').defaultRandom().primaryKey(),
  userId: text('user_id').notNull().unique(),
  name: text('name').notNull(),
  email: text('email').notNull().unique(),
  imageUrl: text('image_url'),
  createdAt: timestamp('created_at').defaultNow().notNull(),
});

// 2. Рабочие пространства кампаний
export const campaigns = pgTable('campaigns', {
  id: uuid('id').defaultRandom().primaryKey(),
  userId: text('user_id').notNull(),
  name: text('name').notNull(),
  icon: text('icon'),
  officeStyle: text('office_style').default('wooden').notNull(), // 'wooden' | 'modern'
  createdAt: timestamp('created_at').defaultNow().notNull(),
  updatedAt: timestamp('updated_at').defaultNow().notNull(),
});

// 3. Стратегические планы Master Agent
export const agentPlans = pgTable('agent_plans', {
  id: uuid('id').defaultRandom().primaryKey(),
  campaignId: text('campaign_id').notNull(),
  userId: text('user_id').notNull(),
  title: text('title').notNull(),
  goal: text('goal'),
  status: text('status').default('active').notNull(), // 'active' | 'completed' |
'cancelled'
  createdAt: timestamp('created_at').defaultNow().notNull(),
  updatedAt: timestamp('updated_at').defaultNow().notNull(),
});

// 4. Подзадачи субагентов
export const agentTasks = pgTable('agent_tasks', {
  id: uuid('id').defaultRandom().primaryKey(),
  planId: uuid('plan_id').references(() => agentPlans.id),
  campaignId: text('campaign_id').notNull(),
```

```
  agentType: text('agent_type').notNull(), // 'researcher' | 'creator' | 'scheduler' |
'analytics'
  title: text('title').notNull(),
  instruction: text('instruction').notNull(),
  status: text('status').default('pending').notNull(), // 'pending' | 'running' |
'completed' | 'failed' | 'cancelled' | 'waiting_for_user'
  dependsOn: uuid('depends_on'), // Явная ссылка на родительскую задачу
  output: jsonb('output'), // Структурированный JSON-результат субагента
  metadata: jsonb('metadata'), // Метаданные (например, streamUrl для E2B или промпт 2FA)

  // Параметры фонового планировщика Inngest
  nextRun: timestamp('next_run'),
  repeatInterval: text('repeat_interval'), // 'daily' | 'weekly' и т.д.
  until: timestamp('until'),
  runCount: integer('run_count').default(0).notNull(),

  createdAt: timestamp('created_at').defaultNow().notNull(),
  updatedAt: timestamp('updated_at').defaultNow().notNull(),
});

// 5. Журнал аудита выполнения задач (Audit Trail)
export const agentTaskRuns = pgTable('agent_task_runs', {
  id: uuid('id').defaultRandom().primaryKey(),
  taskId: uuid('task_id').references(() => agentTasks.id, { onDelete: 'cascade'
}).notNull(),
  attempt: text('attempt'),
  status: text('status').notNull(),
  output: jsonb('output'),
  error: text('error'),
  logs: jsonb('logs'),
  startedAt: timestamp('started_at').defaultNow().notNull(),
  completedAt: timestamp('completed_at'),
});

// 6. Диалоги пользователя и Master Agent
export const agentMessages = pgTable('agent_messages', {
  id: uuid('id').defaultRandom().primaryKey(),
  campaignId: text('campaign_id').notNull(),
  sender: text('sender').notNull(), // 'user' | 'master' | 'researcher' | 'creator' |
'scheduler' | 'analytics'
  content: text('content').notNull(),
  metadata: jsonb('metadata'),
  createdAt: timestamp('created_at').defaultNow().notNull(),
});

// 7. Межагентные логи коммуникаций
export const agentCommunications = pgTable('agent_communications', {
  id: uuid('id').defaultRandom().primaryKey(),
  planId: uuid('plan_id').references(() => agentPlans.id),
  fromAgent: text('from_agent').notNull(),
  toAgent: text('to_agent').notNull(),
  message: text('message').notNull(),
  payload: jsonb('payload'),
  createdAt: timestamp('created_at').defaultNow().notNull(),
});
```

3. СИСТЕМНЫЕ ПРОМПТЫ И ОПРЕДЕЛЕНИЯ АГЕНТОВ (OPENAI AGENT SDK)

Системные промпты субагентов строятся по четырехкомпонентной структуре: **Роль**, **Обязанности**, **Правила применения инструментов (Tools)** и **Формат ответа (JSON)**.

3.1 Creator Agent (`lib/ai/creator-agent.ts`)

```
import { Agent } from "@openai/agent-sdk";
import { generateImageTool, transformImageTool, saveDraftTool } from "@/lib/ai/tools";

export const creatorAgentSystemPrompt = `
Ты — эксперт по контент-маркетингу и визуальный дизайнер в команде ИИ-агентов.
Твоя главная задача — создавать готовые посты для социальных сетей (Instagram, TikTok,
YouTube, X) на основе данных от Research Agent.

### Твои обязанности:
1. Копирайтинг: Писать вовлекающие посты с цепляющим заголовком (Hook), структурированным
текстом (Body), призывом к действию (CTA) и релевантными хэштегами.
2. Визуализация: Генерировать изображения с помощью библиотеки generateImage или
трансформировать готовые визуалы через transformImage.
3. Форматирование: Учитывать лимиты и стиль целевой платформы.

### Правила работы с инструментами (Tools):
- Если нужен новый визуал, сформулируй подробный промпт на английском языке и вызови
generateImage.
- Для редактирования изображения (удаление фона, изменение размера) используй
transformImage.
- Обязательно сохраняй черновик с помощью saveDraftTool.

### Формат ответа (JSON):
Возвращай результат строго в виде структурированного JSON-объекта:
{
  "caption": "Текст поста с эмодзи и переносами строк.",
  "hashtags": ["hashtag1", "hashtag2"],
  "imageUrl": "https://ik.imagekit.io/...",
  "platform": "instagram"
}
`;

export const creatorAgent = new Agent({
  name: "Creator Agent",
  instructions: creatorAgentSystemPrompt,
  model: process.env.OPENAI_DEFAULT_MODEL || "gpt-4o-mini",
  tools: [generateImageTool, transformImageTool, saveDraftTool],
});
```

3.2 Research Agent (`lib/ai/research-agent.ts`)

```
import { Agent } from "@openai/agent-sdk";
import { webSearchTool, saveResearchTool } from "@/lib/ai/tools";

export const researchAgentSystemPrompt = `
```

```
Ты — эксперт по маркетинговым исследованиям и анализу трендов в социальных сетях.
Твоя цель — находить актуальные темы, идеи постов, хэштеги и анализировать конкурентов.

### Твои обязанности:
1. Поиск трендов: Исследовать веб-страницы и соцсети.
2. Анализ конкурентов: Собирать ключевые тезисы и формат визуалов.
3. Отчетность: Формировать структурированные данные для Creator Agent.

### Правила использования инструментов:
- Используй webSearch для быстрого поиска новостей.
- Если требуется интерактивный браузер, парсинг страниц или заход на защищенные сайты,
используй песочницу E2B Sandbox.
- Сохраняй результаты через saveResearchTool.

### Формат ответа (JSON):
{
  "topics": [{"title": "Тема 1", "summary": "Описание"}],
  "keyInsights": ["Вывод 1", "Вывод 2"],
  "recommendedAngles": ["Ракурс подачи 1"],
  "sources": ["https://example.com"]
}
`;

export const researchAgent = new Agent({
  name: "Research Agent",
  instructions: researchAgentSystemPrompt,
  model: process.env.OPENAI_DEFAULT_MODEL || "gpt-4o-mini",
  tools: [webSearchTool, saveResearchTool],
});
```

3.3 Scheduler Agent (`lib/ai/scheduler-agent.ts`)

```
import { Agent } from "@openai/agent-sdk";
import { blottoPublishTool, blottoCheckAccountTool } from "@/lib/ai/tools";

export const schedulerAgentSystemPrompt = `
Ты — контент-менеджер и специалист по публикации в социальных сетях.
Твоя цель — брать черновики постов от Creator Agent и отправлять их в соцсети через
Blotato API.

### Твои обязанности:
1. Проверка времени: Сопоставлять расписание с часовыми поясами.
2. Автопостинг: Вызывать инструменты Blotato MCP для отправки текста и медиа.
3. Обновление статусов: Фиксировать результат отправки в БД.

### Инструкция:
- Вызывай blottoPublishTool для моментальной или отложенной выгрузки.
- Проверяй подключенные аккаунты через blottoCheckAccountTool.

### Формат ответа (JSON):
{
  "scheduledPostId": "blt_12345",
  "publicationTime": "2026-11-01T08:00:00Z",
  "platform": "instagram",
  "status": "scheduled"
}
```

```
`;

export const schedulerAgent = new Agent({
  name: "Scheduler Agent",
  instructions: schedulerAgentSystemPrompt,
  model: process.env.OPENAI_DEFAULT_MODEL || "gpt-4o-mini",
  tools: [blottoPublishTool, blottoCheckAccountTool],
});
```

3.4 Analytics Agent (`lib/ai/analytics-agent.ts`)

```
import { Agent } from "@openai/agent-sdk";
import { blottoGetAnalyticsTool } from "@/lib/ai/tools";

export const analyticsAgentSystemPrompt = `
Ты — аналитик данных и специалист по оценке эффективности контента.
Твоя цель — собирать метрики по опубликованным материалам и давать рекомендации.

### Обязанности:
1. Сбор статистики: Запрашивать просмотры, лайки, комментарии, охваты.
2. Оценка KPI: Оправдывает ли контент поставленные цели.
3. Корректировка: Передавать рекомендации Master Agent.

### Формат ответа (JSON):
{
  "periodDays": 7,
  "totalEngagement": 14500,
  "topPerformingPosts": ["post_id_1"],
  "recommendations": ["Увеличить количество видеоконтента"]
}
`;

export const analyticsAgent = new Agent({
  name: "Analytics Agent",
  instructions: analyticsAgentSystemPrompt,
  model: process.env.OPENAI_DEFAULT_MODEL || "gpt-4o-mini",
  tools: [blottoGetAnalyticsTool],
});
```

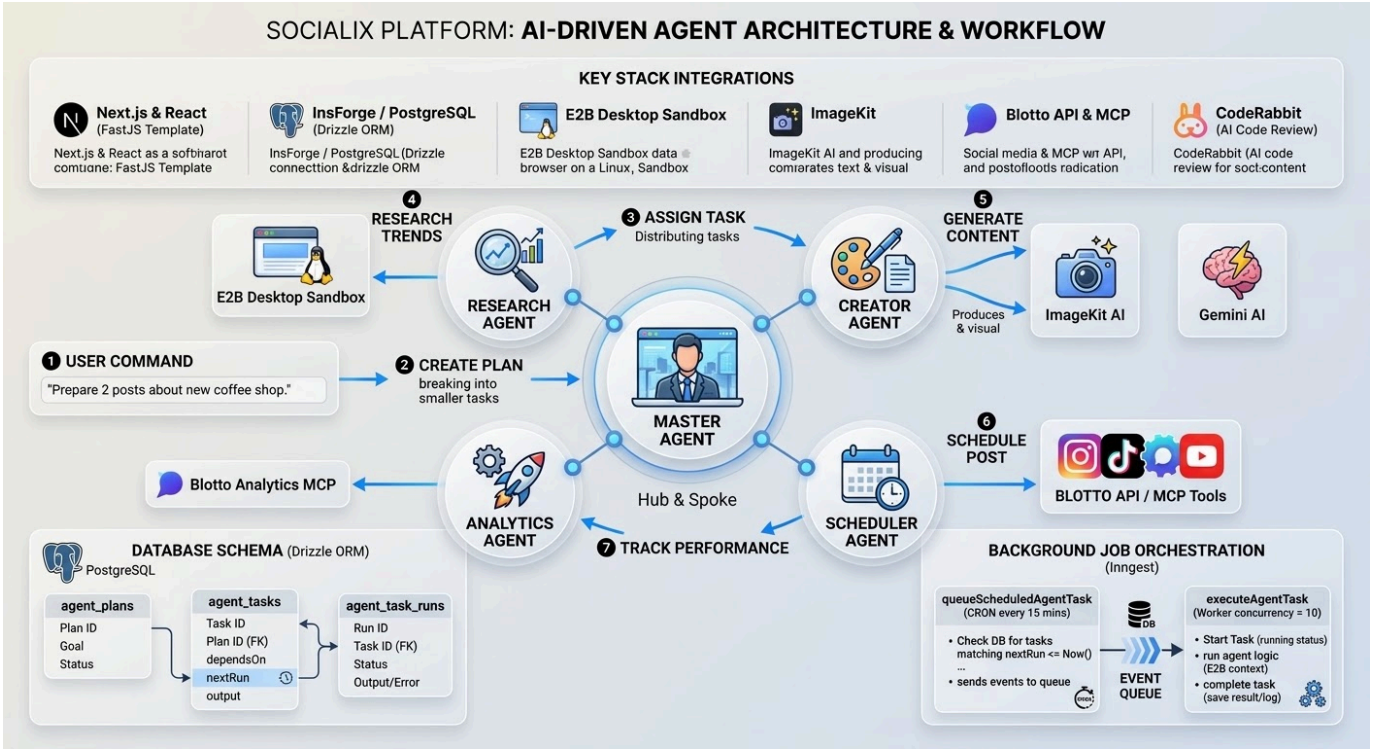
---

4. МЕЖАГЕНТНАЯ ПЕРЕДАЧА КОНТЕКСТА И ИСПОЛНЕНИЕ ЦЕПОЧКИ (HUB & SPOKE IN CODE)

Передача результатов между агентами происходит через сохранение `output` предыдущей задачи в PostgreSQL и передачу его во входной промпт следующего субагента.

4.1 Жизненный цикл задачи (AI Multi-Agent Task Lifecycle)

Графическая схема жизненного цикла задачи



Подробное текстовое описание 10 этапов жизненного цикла:

- **Step 1 & 2 (User Request & Master Decomposition):** Пользователь передает промпт в UI. Master Agent анализирует его, создает запись в `agent_plans` и декомпозирует задачу на подзадачи со связями `dependsOn`.
- **Step 3 & 4 (Persistence & Queue Dispatch):** Подзадачи записываются в таблицу `agent_tasks` со статусом `pending`, и Inngest генерирует событие `agent/task.execute`.
- **Step 5 (Task Pickup & Context Hydration):** Inngest Worker извлекает задачу, переводит статус в `running` и запрашивает `output` предшествующих завершенных задач из PostgreSQL.
- **Step 6 (Specialized Sub-Agent Execution):** Исполняется нужный субагент через OpenAI Agent SDK с вызовом соответствующих инструментов (E2B Sandbox, ImageKit, Blotato MCP).
- **Step 7 & 8 (Result Persistence & Cascade Triggering):** Результат сохраняется в формате JSONB в поле `agent_tasks.output`, и Inngest запускает следующие зависимые задачи.
- **Step 9 & 10 (Master Agent Synthesis & User Notification):** Master Agent считывает выполнение всей цепочки, формирует сводный отчет и передает его пользователю в веб-интерфейсе.

4.2 Master Agent Planner (`lib/ai/master-agent-planner.ts`)

```
import { db } from "@/db";
import { agentPlans, agentTasks } from "@/db/schema";
import { inngest } from "@/lib/inngest/client";

export async function createCampaignWorkflow(userId: string, campaignId: string,
userPrompt: string) {
  // 1. Создание стратегического плана
  const [plan] = await db.insert(agentPlans).values({
    campaignId,
    userId,
    title: "Кампания для Instagram",
```

```
    goal: userPrompt,
    status: "active",
  }).returning();

  // 2. Декомпозиция на последовательные подзадачи с зависимостями

  // Задача 1: Исследование (Research Agent)
  const [researchTask] = await db.insert(agentTasks).values({
    planId: plan.id,
    campaignId,
    agentType: "researcher",
    title: "Поиск трендов по теме",
    instruction: "Найди 3 актуальных тренда по теме: " + userPrompt,
    status: "pending",
  }).returning();

  // Задача 2: Генерация контента (Creator Agent) - зависит от задачи 1
  const [creatorTask] = await db.insert(agentTasks).values({
    planId: plan.id,
    campaignId,
    agentType: "creator",
    title: "Написание текста и создание визуала",
    instruction: "Создай пост для Instagram на основе найденных трендов",
    status: "pending",
    dependsOn: researchTask.id, // Явная связь в БД
  }).returning();

  // 3. Отправка первой задачи в очередь Inngest
  await inngest.send({
    name: "agent/task.execute",
    data: { taskId: researchTask.id },
  });

  return { planId: plan.id, tasks: [researchTask.id, creatorTask.id] };
}
```

4.3 Исполнитель логики субагентов (`lib/ai/agent-runner.ts`)

```
import { researchAgent } from "./research-agent";
import { creatorAgent } from "./creator-agent";
import { schedulerAgent } from "./scheduler-agent";
import { analyticsAgent } from "./analytics-agent";

export async function runSubAgent({
  agentType,
  instruction,
  contextFromPreviousAgent,
  sandbox
}: {
  agentType: string;
  instruction: string;
  contextFromPreviousAgent?: any;
  sandbox?: any;
}) {
  // Формируем расширенный промпт с передачей контекста предшественника
  let fullPrompt = instruction;
```

```
  if (contextFromPreviousAgent) {
    fullPrompt += `\n\n[Результаты и данные от предыдущего
агента]:\n${JSON.stringify(contextFromPreviousAgent, null, 2)}`;
  }

  switch (agentType) {
    case "researcher": {
      const response = await researchAgent.run(fullPrompt);
      return response.output;
    }
    case "creator": {
      const response = await creatorAgent.run(fullPrompt);
      return response.output;
    }
    case "scheduler": {
      const response = await schedulerAgent.run(fullPrompt);
      return response.output;
    }
    case "analytics": {
      const response = await analyticsAgent.run(fullPrompt);
      return response.output;
    }
    default:
      throw new Error(`Неизвестный тип субагента: ${agentType}`);
  }
}
```

4.4 Агрегатор отчетов для Master Agent (`lib/ai/master-agent-reporter.ts`)

```
import { db } from "@/db";
import { agentTasks, agentPlans } from "@/db/schema";
import { eq } from "drizzle-orm";

export async function getMasterAgentFinalReport(planId: string) {
  const [plan] = await db.select().from(agentPlans).where(eq(agentPlans.id, planId));
  const tasks = await db.select().from(agentTasks).where(eq(agentTasks.planId, planId));

  const isAllCompleted = tasks.every((t) => t.status === "completed");

  if (!isAllCompleted) {
    return {
      status: "in_progress",
      completedTasks: tasks.filter((t) => t.status === "completed").length,
      totalTasks: tasks.length,
    };
  }

  const summaryReport = tasks.map((task) => ({
    agentRole: task.agentType,
    taskTitle: task.title,
    executionResult: task.output,
  }));

  return {
    status: "completed",
    campaignTitle: plan.title,
```

```
    report: summaryReport,
  };
}
```

---

5. ФОНОВАЯ ОРКЕСТРАЦИЯ INNGEST И ИНТЕГРАЦИЯ E2B DESKTOP SANDBOX

Обработка длительных задач, выполнение cron-проверок и изоляция виртуальных контейнеров браузера выполняются в фоновом режиме воркерами Inngest (`lib/inngest/functions.ts`).

5.1 Полный кодовый модуль: `lib/inngest/functions.ts`

```
import { inngest } from "./client";
import { db } from "@/db";
import { agentTasks, agentTaskRuns } from "@/db/schema";
import { eq, lte, and } from "drizzle-orm";
import { Sandbox } from "@e2b/desktop";
import { runSubAgent } from "@/lib/ai/agent-runner";

// 1. Cron-планировщик: проверка отложенных задач каждые 15 минут
export const queueScheduledAgentTasks = inngest.createFunction(
  { id: "queue-scheduled-agent-tasks" },
  { cron: "*/15 * * * *" },
  async ({ step }) => {
    const pendingTasks = await step.run("fetch-pending-tasks", async () => {
      return await db
        .select()
        .from(agentTasks)
        .where(
          and(
            eq(agentTasks.status, "pending"),
            lte(agentTasks.nextRun, new Date())
          )
        );
    });

    const events = pendingTasks.map((task) => ({
      name: "agent/task.execute",
      data: { taskId: task.id },
    }));

    if (events.length > 0) {
      await step.sendEvent("trigger-execution-events", events);
    }

    return { queuedCount: events.length };
  }
);

// 2. Воркер выполнения задач с ограничением параллелизма и E2B Sandbox
export const executeAgentTask = inngest.createFunction(
  {
```

```
    id: "execute-agent-task",
    concurrency: { limit: 10 }
  },
  { event: "agent/task.execute" },
  async ({ event, step }) => {
    const { taskId } = event.data;

    // Шаг 1: Загрузка контекста задачи и ответа предшественника
    const { task, previousOutput } = await step.run("fetch-context", async () => {
      const [currentTask] = await db.select().from(agentTasks).where(eq(agentTasks.id,
taskId));
      if (!currentTask) return { task: null, previousOutput: null };

      let prevOutput = null;
      if (currentTask.dependsOn) {
        const [prevTask] = await db.select().from(agentTasks).where(eq(agentTasks.id,
currentTask.dependsOn));
        prevOutput = prevTask?.output || null;
      }

      // Переводим задачу в статус 'running'
      await db.update(agentTasks).set({ status: "running", updatedAt: new Date()
}).where(eq(agentTasks.id, taskId));

      return { task: currentTask, previousOutput: prevOutput };
    });

    if (!task) return { error: "Task not found" };

    // Шаг 2: Безопасное исполнение в E2B Desktop Sandbox
    const result = await step.run("run-agent-execution", async () => {
      let sandbox: Sandbox | null = null;
      try {
        const requiresBrowser = task.agentType === "researcher" ||
task.instruction.includes("browser");

        if (requiresBrowser) {
          sandbox = await Sandbox.create({
            apiKey: process.env.E2B_API_KEY,
            timeoutMs: 5 * 60 * 1000, // 5 минут
          });
        }

        const output = await runSubAgent({
          agentType: task.agentType,
          instruction: task.instruction,
          contextFromPreviousAgent: previousOutput,
          sandbox,
        });

        return { success: true, output };
      } catch (err: any) {
        return { success: false, error: err.message || "Ошибка исполнения субагента" };
      } finally {
        if (sandbox) {
          await sandbox.kill(); // Гарантированное освобождение ресурсов
        }
      }
```

```
    });

    // Шаг 3: Сохранение результатов и запуск зависимых задач
    await step.run("complete-task-and-trigger-dependents", async () => {
      const finalStatus = result.success ? "completed" : "failed";

      await db
        .update(agentTasks)
        .set({
          status: finalStatus,
          output: result.output || { error: result.error },
          runCount: task.runCount + 1,
          updatedAt: new Date(),
        })
        .where(eq(agentTasks.id, taskId));

      await db.insert(agentTaskRuns).values({
        taskId: task.id,
        status: finalStatus,
        output: result.output || null,
        error: result.error || null,
      });

      // Каскадный запуск следующих задач по графу зависимостей
      if (result.success) {
        const dependentTasks = await
db.select().from(agentTasks).where(eq(agentTasks.dependsOn, taskId));
        for (const nextTask of dependentTasks) {
          await inngest.send({
            name: "agent/task.execute",
            data: { taskId: nextTask.id },
          });
        }
      }
    });

    return { taskId, status: result.success ? "completed" : "failed" };
  }
);
```

---

6. ВЗАИМОДЕЙСТВИЕ С ЧЕЛОВЕКОМ (HUMAN-IN-THE-LOOP / HITL)

При обнаружении капчи, формы авторизации или требования 2FA субагент не завершается с ошибкой и не тратит CPU в бесконечном цикле, а временно замораживает шаги Inngest.

6.1 Интерактивная оркестрация Inngest (`lib/inngest/executeInteractiveTask.ts`)

```
import { inngest } from "./client";
import { Sandbox } from "@e2b/code-interpreter";
import { db } from "@/db";
import { agentTasks } from "@/db/schema";
import { eq } from "drizzle-orm";
```

```
export const executeInteractiveTask = inngest.createFunction(
  { id: "execute-interactive-task" },
  { event: "agent/research.requested" },
  async ({ event, step }) => {
    const { taskId, instruction } = event.data;

    await step.run("mark-task-running", async () => {
      await db.update(agentTasks).set({ status: "running", updatedAt: new Date()
}).where(eq(agentTasks.id, taskId));
    });

    // Первичный запуск
    const executionResult = await step.run("run-e2b-step", async () => {
      const sandbox = await Sandbox.create();
      try {
        // Симуляция проверки на 2FA/капчу
        const needsHuman = true; // Условный флаг от Vision LLM
        if (needsHuman) {
          return {
            status: "NEEDS_USER_INPUT",
            sandboxId: sandbox.sandboxId, // НЕ ЗАКРЫВАЕМ sandbox!
            prompt: "Введите код подтверждения из SMS/2FA:",
          };
        }
        return { status: "SUCCESS", data: {} };
      } catch (err) {
        await sandbox.kill();
        throw err;
      }
    });

    // Если требуется вмешательство человека
    if (executionResult.status === "NEEDS_USER_INPUT") {
      await step.run("notify-ui-waiting", async () => {
        await db.update(agentTasks)
          .set({
            status: "waiting_for_user",
            metadata: { prompt: executionResult.prompt, sandboxId:
executionResult.sandboxId }
          })
          .where(eq(agentTasks.id, taskId));
      });

      // ЗАМОРОЗКА ШАГА INNGEST (до 10 минут, воркер не потребляет RAM/CPU)
      const userResponseEvent = await step.waitForEvent("wait-for-user-response", {
        event: "agent/user.responded",
        timeout: "10m",
        match: "data.taskId",
      });

      // Обработка таймаута
      if (!userResponseEvent) {
        await step.run("handle-timeout", async () => {
          const sandbox = await Sandbox.connect(executionResult.sandboxId);
          await sandbox.kill();
          await db.update(agentTasks)
            .set({ status: "failed", metadata: { error: "Таймаут ожидания ввода
пользователя" } })
```

```
            .where(eq(agentTasks.id, taskId));
        });
        return { status: "TIMEOUT" };
      }

      // РАЗМОРАЖИВАНИЕ: Подключаемся к той же песочнице по sandboxId
      const userInputCode = userResponseEvent.data.code;
      const finalResult = await step.run("resume-e2b-execution", async () => {
        const sandbox = await Sandbox.connect(executionResult.sandboxId);
        try {
          await sandbox.commands.run(`node inject_input.js --input="${userInputCode}"`);
          return { status: "COMPLETED", data: { result: "Авторизация успешно пройдена" }
};
        } finally {
          await sandbox.kill(); // Обязательная очистка
        }
      });

      await step.run("save-final-result", async () => {
        await db.update(agentTasks)
          .set({ status: "completed", output: finalResult.data })
          .where(eq(agentTasks.id, taskId));
      });

      return finalResult;
    }

    return executionResult;
  }
);
```

6.2 API-обработчик разблокировки задачи (`app/api/tasks/respond/route.ts`)

```
import { NextResponse } from "next/server";
import { inngest } from "@/lib/inngest/client";

export async function POST(req: Request) {
  const { taskId, code } = await req.json();

  // Отправка события возобновления, совпадающего с match: "data.taskId"
  await inngest.send({
    name: "agent/user.responded",
    data: { taskId, code },
  });

  return NextResponse.json({ success: true });
}
```

---

7. LIVE-СТРИМИНГ ЭКРАНА ВОРКЕРА E2B В REACT UI

Для визуального отображения работы виртуального браузера в реальном времени используется запуск встроенного VNC/WebSocket-сервера в песочнице E2B и трансляция через `<iframe>`.

7.1 Бэкенд-запуск стриминга (`lib/agent-runner.ts`)

```
import { Sandbox } from "@e2b/desktop";
import { db } from "@/db";
import { agentTasks } from "@/db/schema";
import { eq } from "drizzle-orm";

export async function runDesktopResearchAgent(taskId: string, instruction: string) {
  const desktop = await Sandbox.create();
  try {
    await desktop.launch("google-chrome");

    // Старт веб-трансляции экрана песочницы
    await desktop.stream.start({ requireAuth: false });
    const streamUrl = desktop.stream.getUrl();

    // Запись URL стрима в метаданные задачи для отображения на фронтенде
    await db.update(agentTasks)
      .set({
        status: "running",
        metadata: { streamUrl, sandboxId: desktop.sandboxId }
      })
      .where(eq(agentTasks.id, taskId));

    await desktop.write("https://google.com");
    await desktop.press("Enter");
    await desktop.wait(3000);

    return { status: "SUCCESS" };
  } catch (error) {
    console.error("Agent execution error:", error);
    throw error;
  }
}
```

7.2 UI-компонент с вкладками (`components/AgentTaskWorkspace.tsx`)

```
"use client";

import React, { useState } from "react";
import { Monitor, Terminal, CheckCircle2, RefreshCw } from "lucide-react";

interface TaskViewProps {
  task: {
    id: string;
    status: string;
    metadata?: { streamUrl?: string };
    output?: any;
    logs?: string[];
  };
}

export function AgentTaskWorkspace({ task }: TaskViewProps) {
  const [activeTab, setActiveTab] = useState<"stream" | "logs" | "output">("stream");
```

```
  return (
    <div className="w-full max-w-5xl mx-auto border rounded-xl shadow-lg bg-background
overflow-hidden">
      {/* Шапка вкладок */}
      <div className="flex items-center justify-between border-b px-4 bg-muted/40">
        <div className="flex gap-1">
          <button
            onClick={() => setActiveTab("stream")}
            className={`flex items-center gap-2 px-4 py-3 text-sm font-medium border-b-2
transition-colors ${
              activeTab === "stream" ? "border-primary text-primary bg-background" :
"border-transparent text-muted-foreground"
            }`}
          >
            <Monitor className="w-4 h-4" />
            Экран браузера (Live)
            {task.status === "running" && (
              <span className="flex h-2 w-2 relative">
                <span className="animate-ping absolute inline-flex h-full w-full rounded-
full bg-green-400 opacity-75"></span>
                <span className="relative inline-flex rounded-full h-2 w-2 bg-green-500">
</span>
              </span>
            )}
          </button>
          <button
            onClick={() => setActiveTab("logs")}
            className={`flex items-center gap-2 px-4 py-3 text-sm font-medium border-b-2
transition-colors ${
              activeTab === "logs" ? "border-primary text-primary bg-background" :
"border-transparent text-muted-foreground"
            }`}
          >
            <Terminal className="w-4 h-4" />
            Логи агента
          </button>
          <button
            onClick={() => setActiveTab("output")}
            className={`flex items-center gap-2 px-4 py-3 text-sm font-medium border-b-2
transition-colors ${
              activeTab === "output" ? "border-primary text-primary bg-background" :
"border-transparent text-muted-foreground"
            }`}
          >
            <CheckCircle2 className="w-4 h-4" />
            Результат
          </button>
        </div>
        <div className="text-xs font-mono uppercase px-2 py-1 rounded bg-secondary">
          {task.status}
        </div>
      </div>

      {/* Контент активной вкладки */}
      <div className="p-4 min-h-[500px] flex flex-col justify-center items-center bg-zinc-
950 text-white">
        {activeTab === "stream" && (
          <div className="w-full h-[550px] relative rounded-lg overflow-hidden border
```

```
border-zinc-800 bg-black">
            {task.metadata?.streamUrl ? (
              <iframe
                src={task.metadata.streamUrl}
                className="w-full h-full border-none"
                title="E2B Desktop Live Stream"
                allow="autoplay; clipboard-read; clipboard-write"
              />
            ) : (
              <div className="flex flex-col items-center justify-center h-full text-zinc-
500 gap-3">
                <RefreshCw className="w-8 h-8 animate-spin text-primary" />
                <p>Инициализация виртуального экрана E2B...</p>
              </div>
            )}
          </div>
        )}

        {activeTab === "logs" && (
          <div className="w-full h-[550px] p-4 font-mono text-sm bg-black text-green-400
rounded-lg overflow-y-auto border border-zinc-800">
            <p className="text-zinc-500">[System] Инициализация сессии Sandbox...</p>
            <p>[Agent] Запуск виртуального Chrome</p>
            {task.logs?.map((log, index) => <p key={index}>{log}</p>)}
          </div>
        )}

        {activeTab === "output" && (
          <div className="w-full h-[550px] p-6 bg-zinc-900 rounded-lg overflow-y-auto
text-zinc-200">
            <h3 className="text-lg font-bold mb-4">Собранные данные:</h3>
            <pre className="bg-black p-4 rounded text-xs overflow-x-auto">
              {JSON.stringify(task.output || { message: "Обработка..." }, null, 2)}
            </pre>
          </div>
        )}
      </div>
    </div>
  );
}
```

---

8. ИНТЕГРАЦИИ ВНЕШНИХ СЕРВИСОВ И КАСТОМНЫЕ ИНСТРУМЕНТЫ (TOOLS)

8.1 Инструменты ImageKit AI (`lib/ai/imagekit-tools.ts`)

```
import { tool } from "@openai/agent-sdk";
import { z } from "zod";
import ImageKit from "imagekit";

const imagekit = new ImageKit({
  publicKey: process.env.IMAGEKIT_PUBLIC_KEY!,
```

```
  privateKey: process.env.IMAGEKIT_PRIVATE_KEY!,
  urlEndpoint: process.env.IMAGEKIT_URL_ENDPOINT!,
});

export const generateImageTool = tool({
  name: "generateImage",
  description: "Генерирует новое AI-изображение для поста на основе текстового промпта.",
  parameters: z.object({
    prompt: z.string().describe("Подробный англоязычный промпт."),
    aspectRatio: z.enum(["1:1", "16:9", "9:16"]).default("1:1"),
  }),
  execute: async ({ prompt, aspectRatio }) => {
    // Симуляция генерации и выгрузки в хранилище ImageKit
    const uploadResponse = await imagekit.upload({
      file:
"
DhgGAWjR9awAAAABJRU5ErkJggg==",
      fileName: `generated_${Date.now()}.png`,
      folder: "/agent_posts",
    });

    return {
      success: true,
      imageUrl: uploadResponse.url,
      fileId: uploadResponse.fileId,
    };
  },
});

export const transformImageTool = tool({
  name: "transformImage",
  description: "Применяет AI-трансформации к изображению (bg-remove, resize, overlay).",
  parameters: z.object({
    imageUrl: z.string().url(),
    removeBackground: z.boolean().optional(),
    width: z.number().optional(),
    height: z.number().optional(),
    overlayText: z.string().optional(),
  }),
  execute: async ({ imageUrl, removeBackground, width, height, overlayText }) => {
    const tr: string[] = [];
    if (width || height) tr.push(`w-${width || "auto"},h-${height || "auto"}`);
    if (removeBackground) tr.push("bg-remove");
    if (overlayText) tr.push(`l-text,i-${encodeURIComponent(overlayText)},fs-28,co-
FFFFFF,l-end`);

    const transformedUrl = `${imageUrl}?tr=${tr.join(",")}`;
    return { success: true, transformedUrl };
  },
});
```

8.2 Инструменты Blotato (Blotto) MCP (`lib/ai/blotato-tools.ts`)

```
import { tool } from "@openai/agent-sdk";
import { z } from "zod";
import axios from "axios";
```

```
const BLOTATO_API_URL = process.env.BLOTTO_API_ENDPOINT || "https://api.blotato.com/v1";

async function callBlotatoApi(endpoint: string, method: "GET" | "POST" = "GET", body?:
any) {
  const response = await axios({
    url: `${BLOTATO_API_URL}${endpoint}`,
    method,
    headers: {
      Authorization: `Bearer ${process.env.BLOTTO_API_KEY}`,
      "Content-Type": "application/json",
    },
    data: body,
  });
  return response.data;
}

export const blottoCheckAccountTool = tool({
  name: "blottoCheckAccount",
  description: "Проверяет подключение аккаунтов социальных сетей.",
  parameters: z.object({
    platform: z.string().describe("Платформа: instagram, tiktok, youtube, x"),
  }),
  execute: async ({ platform }) => {
    try {
      const accounts = await callBlotatoApi(`/accounts?platform=${platform}`);
      return { success: true, connected: accounts.length > 0, accounts };
    } catch (error: any) {
      return { success: false, error: error.message };
    }
  },
});

export const blottoPublishTool = tool({
  name: "blottoPublish",
  description: "Опубликовать или запланировать пост через Blotato API.",
  parameters: z.object({
    platforms: z.array(z.string()).describe("Список платформ"),
    caption: z.string().describe("Текст поста с хэштегами"),
    mediaUrl: z.string().url().optional(),
    scheduleTime: z.string().optional().describe("ISO строка даты"),
  }),
  execute: async ({ platforms, caption, mediaUrl, scheduleTime }) => {
    try {
      const res = await callBlotatoApi("/posts", "POST", {
        platforms,
        content: { caption, mediaUrl },
        schedule: scheduleTime || null,
      });
      return {
        success: true,
        postId: res.id,
        status: res.status,
        scheduledFor: res.scheduledAt || null,
      };
    } catch (error: any) {
      return { success: false, error: error.response?.data?.message || error.message };
    }
```

```
  },
});

export const blottoGetAnalyticsTool = tool({
  name: "blottoGetAnalytics",
  description: "Запросить аналитику по опубликованным постам.",
  parameters: z.object({
    postId: z.string().describe("ID поста в системе Blotato"),
  }),
  execute: async ({ postId }) => {
    try {
      const res = await callBlotatoApi(`/posts/${postId}/analytics`);
      return { success: true, metrics: res };
    } catch (error: any) {
      return { success: false, error: error.message };
    }
  },
});
```

8.3 Контроллер вебхуков Blotato (`app/api/webhooks/blotto/route.ts`)

```
import { NextResponse } from "next/server";
import { db } from "@/db";
import { agentTasks, agentTaskRuns } from "@/db/schema";
import { eq } from "drizzle-orm";

export async function POST(req: Request) {
  try {
    const body = await req.json();
    const { taskId, postId, status, platform, error } = body;

    if (!taskId) {
      return NextResponse.json({ error: "Missing taskId parameter" }, { status: 400 });
    }

    const isSuccess = status === "published" || status === "success";
    const finalStatus = isSuccess ? "completed" : "failed";

    await db
      .update(agentTasks)
      .set({
        status: finalStatus,
        output: { postId, platform, status, receivedAt: new Date().toISOString() },
        updatedAt: new Date(),
      })
      .where(eq(agentTasks.id, taskId));

    await db.insert(agentTaskRuns).values({
      taskId,
      status: finalStatus,
      output: { postId, platform },
      error: error || null,
      completedAt: new Date(),
    });

    return NextResponse.json({ success: true, taskId });
```

```
  } catch (err: any) {
    return NextResponse.json({ error: err.message }, { status: 500 });
  }
}
```

---

9. ФРОНТЕНД И ИНТЕРАКТИВНЫЙ 2D-ОФИС (REACT + SWR POLLING)

9.1 SWR-Эндпоинт опроса активных задач (`app/api/agent-tasks/active/route.ts`)

```
import { NextResponse } from 'next/server';
import { db } from '@/db';
import { agentTasks } from '@/db/schema';
import { eq, or, desc } from 'drizzle-orm';

export async function GET() {
  const tasks = await db.select().from(agentTasks)
    .where(or(eq(agentTasks.status, 'running'), eq(agentTasks.status, 'pending')))
    .orderBy(desc(agentTasks.updatedAt));

  const map: Record<string, any> = {
    researcher: { status: 'idle' },
    creator: { status: 'idle' },
    scheduler: { status: 'idle' },
    analytics: { status: 'idle' },
  };

  tasks.forEach((t) => {
    map[t.agentType] = { status: t.status, title: t.title };
  });

  return NextResponse.json(map, { headers: { 'Cache-Control': 'no-store' } });
}
```

9.2 Интерактивный компонент 2D-офиса (`components/OfficeStudio.tsx`)

```
"use client";

import React, { useState, useEffect } from "react";
import Image from "next/image";

interface Seat {
  id: string;
  x: number;
  y: number;
  character: {
    name: string;
    animations: { typing: string[] };
  };
}

interface OfficeStudioProps {
```

```
  sceneBundle: {
    office: { background: string; width: number; height: number };
    seats: Seat[];
  };
  activeTasks?: Record<string, { status: string; title?: string }>;
}

export function OfficeStudio({ sceneBundle, activeTasks }: OfficeStudioProps) {
  const [frameIndex, setFrameIndex] = useState(0);

  // Переключение кадров анимации персонажей
  useEffect(() => {
    const interval = setInterval(() => {
      setFrameIndex((prev) => (prev + 1) % 4);
    }, 250);
    return () => clearInterval(interval);
  }, []);

  return (
    <div className="relative w-full overflow-hidden border rounded-xl bg-slate-950 shadow-
2xl" style={{ aspectRatio: `${sceneBundle.office.width} / ${sceneBundle.office.height}`
}}>
      {/* Фоновая картинка офиса */}
      <Image src={sceneBundle.office.background} alt="AI Office" fill className="object-
cover" />

      {/* Наложение персонажей по X/Y координатам */}
      {sceneBundle.seats.map((seat) => {
        const task = activeTasks?.[seat.id];
        const isWorking = task?.status === "running";
        const frames = seat.character.animations.typing;
        const currentFrame = frames[frameIndex % frames.length] ||
seat.character.animations.typing[0];

        return (
          <div
            key={seat.id}
            className="absolute transition-all duration-300 transform -translate-x-1/2 -
translate-y-1/2 group"
            style={{
              left: `${(seat.x / sceneBundle.office.width) * 100}%`,
              top: `${(seat.y / sceneBundle.office.height) * 100}%`,
            }}
          >
            {/* Статус-плашка над персонажем */}
            <div className={`absolute -top-10 left-1/2 transform -translate-x-1/2 px-2 py-
1 rounded-md text-[10px] font-mono border whitespace-nowrap backdrop-blur-md transition-
all ${
              isWorking ? "bg-amber-950/90 border-amber-500/50 text-amber-300" : "bg-
slate-900/80 border-slate-700 text-slate-400"
            }`}>
              <span className="flex items-center gap-1.5">
                <span className={`w-1.5 h-1.5 rounded-full ${isWorking ? "bg-amber-400
animate-ping" : "bg-slate-500"}`} />
                {isWorking ? task?.title || "Работает..." : "Zzz (Idle)"}
              </span>
            </div>
```

```
            {/* Спрайт персонажа */}
            <div className="relative w-16 h-16">
              <Image src={currentFrame} alt={seat.character.name} fill className="object-
contain" />
            </div>
          </div>
        );
      })}
    </div>
  );
}
```

---

10. БЕЗОПАСНОСТЬ И КОНФИГУРАЦИЯ ДЕПЛОЯ

10.1 Авторизационный Middleware Next.js (`middleware.ts`)

```
import { NextResponse } from 'next/server';
import type { NextRequest } from 'next/server';
import { jwtVerify } from 'jose';

const JWT_SECRET = new TextEncoder().encode(process.env.INSFORGE_JWT_SECRET || 'secret');

export async function middleware(req: NextRequest) {
  const token = req.cookies.get('insforge_token')?.value;
  const isAuthPage = req.nextUrl.pathname.startsWith('/sign-in');

  if (!token && !isAuthPage) {
    return NextResponse.redirect(new URL('/sign-in', req.url));
  }

  try {
    if (token) await jwtVerify(token, JWT_SECRET);
    return NextResponse.next();
  } catch (err) {
    return NextResponse.redirect(new URL('/sign-in', req.url));
  }
}

export const config = {
  matcher: ['/overview/:path*', '/agents/:path*', '/api/agent-tasks/:path*'],
};
```

10.2 Конфигурация Dockerfile Multi-stage

```
FROM node:20-alpine AS builder
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci
COPY . .
ENV NEXT_TELEMETRY_DISABLED 1
RUN npm run build
```

```
FROM node:20-alpine AS runner
WORKDIR /app
ENV NODE_ENV production
ENV NEXT_TELEMETRY_DISABLED 1

RUN addgroup --system --gid 1001 nodejs
RUN adduser --system --uid 1001 nextjs

COPY --from=builder /app/public ./public
COPY --from=builder --chown=nextjs:nodejs /app/.next/standalone ./
COPY --from=builder --chown=nextjs:nodejs /app/.next/static ./.next/static

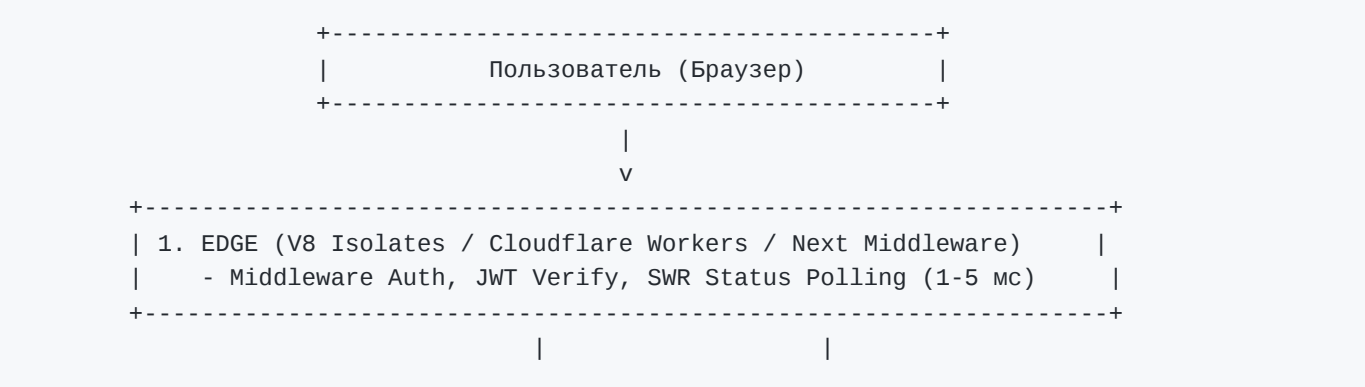
USER nextjs
EXPOSE 3000
ENV PORT 3000
CMD ["node", "server.js"]
```

10.3 Конфигурация CI/CD CodeRabbit (`.coderabbit.yaml`)

```
version: "2"
language: "ru-RU"
tone_instructions: "Будь строгим и профессиональным техническим ревьюером."
reviews: "auto"
collapse_walkthrough: false
path_instructions:
  - path: "lib/ai/**/*.ts"
    instructions: "Проверяй типы Zod схем и валидацию API-ключей для OpenAI и Blotato."
  - path: "db/schema.ts"
    instructions: "Убедись, что все внешние ключи (references) имеют каскадные правила и
индексы."
chat:
  auto_reply: true
tools:
  ast-grep:
    enabled: true
  typescript:
    enabled: true
```

---

11. МАТРИЦА СРЕД ВЫПОЛНЕНИЯ (RUNTIME ARCHITECTURE MATRIX)



```
                                  v                   v
       +----------------------------------+ +-----------------------------+
       | 2. DOCKER / NODE.JS (Inngest)    | | 3. E2B DESKTOP SANDBOX      |
       |    - Fonovye voerkery Inngest    | |    - Full Linux + Chromium VM |
       |    - OpenAI Agent SDK & C++ Sharp| |    - Web Scraping & Live Stream|
       +----------------------------------+ +-----------------------------+
```

Компонент платформы	Среда выполнения	Причина выбора и ограничения
Middleware & Auth	**Edge (V8 Isolates)**	Холодный старт ~0.5 мс. Проверка JWT токенов без накладных расходов.
SWR Status Polling	**Edge (V8 Isolates)**	Отдача статусов задач с задержкой < 5 мс при частых запросах.
Inngest Workers	**Docker / Node.js 20+**	Гарантия исполнения фоновых задач до 15 минут, повторные попытки (retries).
ImageKit & Sharp AI	**Docker / Node.js 20+**	Использование нативных C++ библиотек (`sharp`), недоступных в V8 Isolates.
Research Agent (Web)	**E2B Desktop Sandbox**	Полноценный Linux-контейнер с Chromium для авто-авторизации, капчи и парсинга.

---

12. ПЕЧАТНЫЕ СТИЛИ (PRINT & PDF CSS)

Данные CSS правила внедряются для сохранения чистого вывода без обрезки при стандартизованной печати:

```
@media print {
  @page {
    size: A4 portrait;
    margin: 1.5cm;
  }
  body {
    background: #ffffff !important;
    color: #000000 !important;
    font-family: Arial, sans-serif;
  }
  pre, code {
    white-space: pre-wrap !important;
    word-break: break-all !important;
    font-family: "Courier New", monospace !important;
    font-size: 8pt !important;
  }
  img {
    max-width: 100% !important;
    height: auto !important;
    page-break-inside: avoid;
```

```
  }
}
```